# Stable Diffusion from Scratch

## A Decision-Maker's Guide

Lecture Notes

CAS BMAI — ETH Zürich

| | |
|---|---|
| **Format:** | 5 interactive sessions, approx. 2.5 hours total |
| **Prerequisites:** | None (ability to click "Run" in Google Colab) |
| **Audience:** | Managers and non-technical leaders |

March 31, 2026

# Contents


**1 Introduction** **2**
- 1.1 The Co-Pilot Metaphor . . . 2
- 1.2 Toy Dataset . . . 2

**2 Session 1: The Raw Ingredients—Images and Text as Numbers** **2**
- 2.1 Images as Tensors . . . 2
- 2.2 Text as Embedding Vectors . . . 3
- 2.3 The Gap . . . 3

**3 Session 2: The Translator—CLIP** **3**
- 3.1 The Core Idea: Contrastive Learning . . . 3
- 3.2 Building a Mini-CLIP . . . 4
- 3.3 Productive Failure: Memorisation vs. Generalisation . . . 4

**4 Session 3: The Engine—Diffusion** **4**
- 4.1 Forward Process: Adding Noise . . . 4
- 4.2 Reverse Process: Denoising . . . 4
- 4.3 The Noise Schedule . . . 5
- 4.4 Productive Failure: Why Architecture Matters . . . 5

**5 Session 4: The Conversation—Cross-Attention** **5**
- 5.1 From Unconditional to Conditional Generation . . . 5
- 5.2 How Cross-Attention Works . . . 6
- 5.3 Visualising Attention . . . 6
- 5.4 Productive Failure: Uniform Attention . . . 6

**6 Session 5: The Road Trip—The Full Pipeline** **6**
- 6.1 Assembling the Pipeline . . . 6
- 6.2 Classifier-Free Guidance . . . 7
- 6.3 Seed Control and Reproducibility . . . 7
- 6.4 Productive Failure: Compositional Complexity . . . 7

**7 The Co-Pilot Reference Card** **8**

**8 Five Questions to Ask an AI Image Generation Vendor** **8**

**9 Vendor Evaluation: Decision Tree** **8**

**10 Jargon Decoder** **9**

**11 Red Flags When Evaluating Vendors** **9**

**12 Conclusion** **9**

# 1 Introduction

Text-to-image generation—the ability to produce photorealistic pictures from a short textual description—has moved from a research curiosity to a strategic capability in less than three years. Products such as DALL-E, Midjourney, and Stable Diffusion now power content pipelines in marketing, e-commerce, entertainment, and product design.

These lecture notes accompany a hands-on course in which participants *build* a miniature text-to-image system themselves, step by step. The system is small enough to train on a laptop yet architecturally faithful to the full-scale models used in production. The goal is not to turn managers into machine-learning engineers but to equip them with a *working mental model* of the technology—one precise enough to evaluate vendors, diagnose failures, and ask engineering teams the right questions.

## 1.1 The Co-Pilot Metaphor

Throughout the course we use a single, consistent analogy that maps every technical component to a familiar concept:

| **Technical Component** | Metaphor |
|---|---|
| **Diffusion model (U-Net)** | The *driver*—knows how to navigate but not where to go |
| **Text prompt** | The *destination* |
| **CLIP text encoder** | The *co-pilot* reading the map |
| **Cross-attention** | The driver asking the co-pilot at every turn |
| **Guidance scale** | How *loudly* the co-pilot speaks |
| **Negative prompt** | The co-pilot saying "NOT that way" |
| **Random seed** | The random road network |

By the end of Session 5 you will have assembled all parts of this "car" and driven it yourself.

## 1.2 Toy Dataset

All experiments use a **programmatically generated** dataset of coloured geometric shapes on a 32×32 pixel canvas:

- **Shapes:** circle, square, triangle
- **Colours:** red, blue, green
- **Extensions (Session 5):** small/large, centred/offset

Because every image is generated with known ground truth, we can measure model quality objectively at every stage—an advantage rarely available with real-world data.

# 2 Session 1: The Raw Ingredients—Images and Text as Numbers

*Approx. 25 minutes. Metaphor addition: "Here are the parts of the car and the map."*

## 2.1 Images as Tensors

A digital image is stored as a three-dimensional grid of numbers—a **tensor** of shape $H \times W \times C$, where $H$ is the height in pixels, $W$ the width, and $C$ the number of colour channels (three for

RGB images: red, green, blue). Each entry is a value between 0 and 1 representing the intensity of one colour at one pixel location.

For the toy dataset, every image is a $32 \times 32 \times 3$ tensor containing $32 \times 32 \times 3 = 3{,}072$ numbers.

### 2.2 Text as Embedding Vectors

Computers cannot process words directly. Instead, every word (or sub-word token) is mapped to a list of numbers called an **embedding vector**. Words with similar meanings end up close together in this numerical space; unrelated words end up far apart.

The notion of "closeness" is quantified by **cosine similarity**: two vectors that point in roughly the same direction have a cosine similarity near 1; orthogonal vectors score 0; opposite vectors score $-1$.

### 2.3 The Gap

At this stage, image tensors and text embeddings live in *completely different* numerical spaces. A raw pixel vector and a word embedding cannot be compared. Bridging this gap is the central problem that the next session addresses.

**Manager's Briefing**

**Why this matters.** When an AI vendor says their model "understands" images and text together, they are claiming to have bridged exactly this gap. The quality of that bridge—how well visual and textual representations are aligned—determines whether the model produces what you asked for or something subtly (or wildly) wrong.

#### Real-World Story: The $180K Print Disaster

A marketing team approved AI-generated product images on screen and sent them directly to a print shop. The images were encoded in sRGB (the standard for screens); the printer expected CMYK. Colour shifts in the converted files ruined an entire print run. *Lesson:* image representations (colour spaces, resolutions, file formats) are not interchangeable, and automated pipelines must account for conversions.

## 3 Session 2: The Translator—CLIP

*Approx. 30 minutes. Metaphor addition: "The co-pilot learns to read the map."*

### 3.1 The Core Idea: Contrastive Learning

**CLIP** (Contrastive Language–Image Pre-training) is a model that learns to place images and their corresponding text descriptions *near each other* in a shared numerical space, while pushing mismatched pairs apart.

Training proceeds as follows:

1. Collect a batch of image–text pairs.
2. Pass each image through an **image encoder** to obtain an image embedding.
3. Pass each text through a **text encoder** to obtain a text embedding.
4. Compute the cosine similarity between every image embedding and every text embedding in the batch.
5. Adjust the encoders so that *matching* pairs score high and *non-matching* pairs score low.

The loss function that drives this adjustment is called **InfoNCE**. Intuitively, it rewards the model for ranking the correct match above all distractors.

**Key Insight**

After training, CLIP provides a *common language* for images and text. You can compare an image to a sentence—or two images, or two sentences—simply by measuring the distance between their embedding vectors.

### 3.2 Building a Mini-CLIP

In the hands-on exercise, participants train a small CLIP model (∼200K parameters) on the toy dataset. The text encoder converts a 12-token vocabulary into embedding vectors; the image encoder converts flattened pixel arrays through a short sequence of transformations. Training takes approximately 15 seconds on a CPU.

### 3.3 Productive Failure: Memorisation vs. Generalisation

When Mini-CLIP is trained on only three image–text pairs it memorises them perfectly—but fails on any new example. This demonstrates a fundamental tension in machine learning: a model must see enough variety during training to learn *general* patterns rather than specific examples.

**Manager's Briefing**

**Vendor question.** "How large and diverse is your training dataset?" A model trained on narrow data will excel on benchmarks that resemble its training set and fail unpredictably elsewhere.

#### Real-World Story: The CLIP Bias Problem

Researchers discovered that CLIP embeddings encode societal biases present in their web-scraped training data. For example, certain professions were more closely associated with specific genders or ethnicities in embedding space. *Lesson:* the bridge between text and images is only as fair as the data used to build it. Bias audits are essential before deployment.

## 4 Session 3: The Engine—Diffusion

*Approx. 35 minutes. Metaphor addition: "The driver learns to navigate by being dropped at random locations."*

### 4.1 Forward Process: Adding Noise

The diffusion framework rests on a surprisingly simple idea. Start with a clean image and gradually add random noise, step by step, until the image becomes pure static. This is the **forward process**—it is easy, deterministic given the noise schedule, and requires no learning.

At each step $t$, the image becomes a little noisier. By the final step, no trace of the original content remains.

### 4.2 Reverse Process: Denoising

The creative act is running this process *in reverse*: starting from pure noise and removing noise step by step until a clean image emerges. A neural network—the **denoiser**—is trained to predict

the noise that was added at each step. Given a noisy image and the current step number, it outputs an estimate of the noise, which is then subtracted.

Repeating this subtraction across all steps (typically 50 in the demo) transforms random static into a coherent image.

### 4.3 The Noise Schedule

The rate at which noise is added (and, conversely, removed) is controlled by a **noise schedule**. The course uses a **cosine schedule**, which adds noise slowly at first, accelerates in the middle, and tapers at the end. The schedule strongly influences image quality: too aggressive and fine details are destroyed before the model can learn them; too gentle and the model wastes capacity on near-clean images.

### 4.4 Productive Failure: Why Architecture Matters

Participants first attempt denoising with a naïve fully connected network (an MLP). This network can learn colours but produces blobby, shapeless outputs. The reason: an MLP treats every pixel independently and has no notion of *spatial neighbourhood*.

The fix is to switch to a **U-Net**—a convolutional architecture that processes local patches and preserves spatial structure through skip connections. The U-Net used in the demo has approximately 150K parameters and trains in under three minutes.

**Key Insight**

The architecture of the denoiser determines *what kinds of structure* the model can learn. Choosing the right architecture is not a minor implementation detail—it is a strategic decision that shapes output quality.

**Manager's Briefing**

**Vendor question.** "What is the architecture of your denoiser, and what are its known limitations?" A model that generates beautiful landscapes may fail at fine text, small faces, or hands—often because of architectural choices in the denoiser.

#### Real-World Story: The Deepfake Dilemma

Diffusion-based generators leave subtle fingerprints in the *frequency domain* of their outputs—patterns invisible to the human eye but detectable by specialised analysis tools. Forensic teams exploit these artefacts to distinguish AI-generated images from photographs. *Lesson:* every generative architecture has a signature; understanding that signature is essential for both quality assurance and content authenticity.

## 5 Session 4: The Conversation—Cross-Attention

*Approx. 30 minutes. Metaphor addition: "At every turn, the driver asks the co-pilot."*

### 5.1 From Unconditional to Conditional Generation

The diffusion model from Session 3 generates images, but it has no way to steer *what* it generates. The driver can navigate but does not know the destination. We need a mechanism for the text prompt (via CLIP embeddings) to influence the denoising process at every step.

### 5.2 How Cross-Attention Works

Cross-attention is the mechanism by which image features "consult" text features during denoising. At each layer of the U-Net where cross-attention is applied:

1. The image features produce a set of **queries** (Q): "What should I look like here?"
2. The text features produce **keys** (K) and **values** (V): "Here is what the prompt says."
3. Each query is compared to all keys via a similarity score.
4. The resulting **attention weights** determine how much each text token influences each spatial location in the image.
5. The weighted combination of values is injected back into the image features.

This process repeats at every denoising step, allowing the text prompt to continuously guide image generation.

**Key Insight**

Cross-attention is not a one-time instruction; it is a *continuous conversation* between the image being generated and the text prompt. At every denoising step and at every spatial location, the image "asks" the text what to do.

### 5.3 Visualising Attention

Attention weights can be visualised as heatmaps showing which parts of the image attend to which words in the prompt. In the demo, participants observe that the word "red" activates spatial locations where the shape is being drawn, while "circle" activates the shape's boundary. This transparency is a powerful diagnostic tool.

### 5.4 Productive Failure: Uniform Attention

When attention weights are forced to be uniform (every text token influences every pixel equally), the output collapses into a blurry, colour-averaged blob. Spatial selectivity—the model's ability to route different words to different image regions—is essential for coherent generation.

**Manager's Briefing**

**Vendor question.** "Can you show me attention maps for a failed generation?" Attention visualisation reveals *why* a model produced a wrong output, not just *that* it did. This is invaluable for debugging and quality assurance.

#### Real-World Story: The Compositional Failure

An e-commerce company used an AI model to generate product images showing "a leather bag on a wooden table." The model sometimes swapped materials—producing a wooden bag on a leather table. This *attribute binding* failure is a known limitation of cross-attention: when multiple objects and attributes are present, the attention mechanism can mis-route attributes to the wrong objects.

## 6 Session 5: The Road Trip—The Full Pipeline

*Approx. 30 minutes. Metaphor: "Start the engine, set the destination, drive."*

### 6.1 Assembling the Pipeline

In this final session, all components are wired together into a complete text-to-image system:

1. The **text prompt** is encoded by the trained Mini-CLIP text encoder into an embedding vector.
2. A **random noise image** is sampled (determined by the seed).
3. Over 50 denoising steps, the **U-Net** progressively removes noise, consulting the text embedding via **cross-attention** at each step.
4. The final output is a clean, generated image matching the prompt.

The climactic moment of the course: participants type a prompt such as "red circle," and watch the system they built transform pure noise into a recognisable red circle over a 50-step filmstrip.

## 6.2 Classifier-Free Guidance

In practice, the denoiser runs twice at each step: once *with* the text prompt and once *without*. The difference between the two outputs indicates the "direction" imposed by the text. This difference is then amplified by a factor called the **guidance scale**.

- **Low guidance** ($\sim$1): the model largely ignores the prompt, producing diverse but potentially off-topic images.
- **High guidance** ($\sim$7–15): the model follows the prompt closely, producing sharp and on-topic results but with less diversity.
- **Very high guidance** ($>$20): outputs become oversaturated and artefact-prone.

In the co-pilot metaphor, the guidance scale controls how loudly the co-pilot speaks. Too quiet and the driver wanders; too loud and the driver oversteers.

## 6.3 Seed Control and Reproducibility

The **random seed** determines the initial noise image. Fixing the seed while varying the prompt allows controlled comparison of how different text inputs steer the same starting point. Conversely, fixing the prompt while varying the seed reveals the range of outputs the model can produce for identical instructions.

**Manager's Briefing**

**Why seeds matter for business.** Reproducibility is essential for auditing, debugging, and regulatory compliance. If a generated image causes a problem, you need to be able to recreate it deterministically. Always require that your AI pipeline logs the seed alongside every generated asset.

## 6.4 Productive Failure: Compositional Complexity

When the dataset is extended from 9 classes (3 shapes $\times$ 3 colours) to 36 classes (adding size and position), the model begins to struggle with novel combinations it has seen rarely during training. This demonstrates the fundamental **combinatorial challenge** of generative models: the number of possible attribute combinations grows multiplicatively, but training data is finite.

### Real-World Story: The Prompt Engineering Economy

Small changes in wording—"a photo of a cat" versus "a professional photograph of a cat"—can produce dramatically different outputs. This sensitivity is not a bug but a direct consequence of how text embeddings work: nearby words occupy nearby points in embedding space, and even small shifts in position alter the guidance signal. *Lesson:* prompt sensitivity is an architectural property, not a user error.

## 7 The Co-Pilot Reference Card

Each session adds one row to a cumulative reference table. By the end of Session 5, participants have a complete cheat sheet:

| **Session** | Component | Metaphor | What It Does |
|---|---|---|---|
| **1** | Images & Text | Car parts & map | Represent visual and textual information as numbers |
| **2** | CLIP | Co-pilot reads map | Aligns image and text representations in a shared space |
| **3** | Diffusion | Driver navigates | Generates images by iteratively removing noise |
| **4** | Cross-Attention | Driver asks co-pilot | Injects text guidance into the image generation process |
| **5** | Full Pipeline | The road trip | Wires all components into a working text-to-image system |

## 8 Five Questions to Ask an AI Image Generation Vendor

1. **Data representation.** What image resolution, colour space, and file format does your model produce—and do these match our downstream requirements?
2. **Training data.** How large and diverse is your training dataset, and what bias audits have been conducted?
3. **Architecture.** What is the architecture of your denoiser, and what are its known failure modes (e.g., text rendering, fine detail, attribute binding)?
4. **Diagnostics.** Can you provide attention maps or other interpretability tools to diagnose failed generations?
5. **Reproducibility.** Does your system log random seeds and model versions for every generated asset, enabling deterministic replay?

## 9 Vendor Evaluation: Decision Tree

When adopting AI image generation, organisations face three broad options:

**API Service**
: Use a third-party API (e.g., OpenAI, Stability AI). Best when: speed to market is paramount, volume is moderate, and customisation needs are low.

**Fine-Tuned Model**
: Take a pre-trained open model and fine-tune it on proprietary data. Best when: domain-specific quality matters (e.g., your brand's visual style) and you have in-house ML capacity.

**Self-Hosted Model**
: Train or deploy a model entirely on your own infrastructure. Best when: data pri-

vacy is non-negotiable, you need full control over the model, and you have significant engineering resources.

The right choice depends on your organisation's constraints along four axes: **data sensitivity**, **customisation depth**, **engineering capacity**, and **cost structure** (per-generation API fees vs. upfront infrastructure investment).

## 10 Jargon Decoder

| Term | Plain-Language Definition |
|---|---|
| **Tensor** | A multi-dimensional grid of numbers (an image is a 3-D tensor) |
| **Embedding** | A list of numbers representing a word, sentence, or image in a form computers can process |
| **Cosine similarity** | A score measuring how similar two embeddings are (1 = identical direction, 0 = unrelated) |
| **CLIP** | A model that learns to place matching images and texts near each other in a shared number space |
| **Contrastive learning** | A training strategy that pushes matches together and mismatches apart |
| **Diffusion** | A generative process that creates images by iteratively removing noise from random static |
| **U-Net** | A neural network architecture with skip connections, well suited for image denoising |
| **Cross-attention** | A mechanism allowing one type of data (images) to query another (text) for guidance |
| **Guidance scale** | A parameter controlling how strongly the text prompt influences image generation |
| **Noise schedule** | The plan governing how quickly noise is added or removed at each step |

## 11 Red Flags When Evaluating Vendors

1. **No bias audit.** The vendor cannot describe what biases exist in their training data or what mitigations are in place.
2. **No reproducibility.** Generated outputs cannot be recreated deterministically (no seed logging, no version tracking).
3. **Black-box failures.** When generation fails, the vendor offers no diagnostic tools (e.g., attention maps) and simply suggests "try a different prompt."
4. **Benchmark-only evaluation.** Quality claims are based solely on standard benchmarks (FID, CLIP score) without evaluation on your domain-specific data.
5. **No content-provenance metadata.** Generated images carry no watermark or C2PA metadata, making it impossible to distinguish them from real photographs downstream.

## 12 Conclusion

The core architecture of text-to-image generation rests on three learnable components—a text–image alignment model (CLIP), a denoising network (U-Net with diffusion), and a conditioning

mechanism (cross-attention)—plus a set of carefully chosen hyper-parameters (noise schedule, guidance scale, seed).

None of these pieces is, on its own, mysterious. Images are grids of numbers. Text is lists of numbers. CLIP aligns the two. Diffusion learns to remove noise. Cross-attention lets text steer the denoiser. The compound system produces results that feel like magic, but every step is auditable, tuneable, and—as you have now seen—buildable from scratch.

Understanding these building blocks transforms the conversation with engineering teams and vendors from "Can the AI do this?" to "Which component is responsible for this limitation, and what would it take to fix it?"